

MODULE-1

BODY DETECTION SYSTEM

CODE USED FOR DEMONSTRATION

```
posedetec.py > ...
1  import cv2
2  import mediapipe as mp
3
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6  mp_draw = mp.solutions.drawing_utils
7  cap = cv2.VideoCapture(0)
8
9  while True:
10     success, frame = cap.read()
11     if not success:
12         break
13     rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
14     results = pose.process(rgb_frame)
15     if results.pose_landmarks:
16         mp_draw.draw_landmarks(
17             frame,
18             results.pose_landmarks,
19             mp_pose.POSE_CONNECTIONS
20         )
21     cv2.imshow("Pose Detection", frame)
22     if cv2.waitKey(1) & 0xFF == ord('q'):
23         break
24 cap.release()
25 cv2.destroyAllWindows()
```

Using OpenCV and MediaPipe, we can access the webcam and use body landmarks to detect the body. The accuracy of the body detection depends on various factors like:

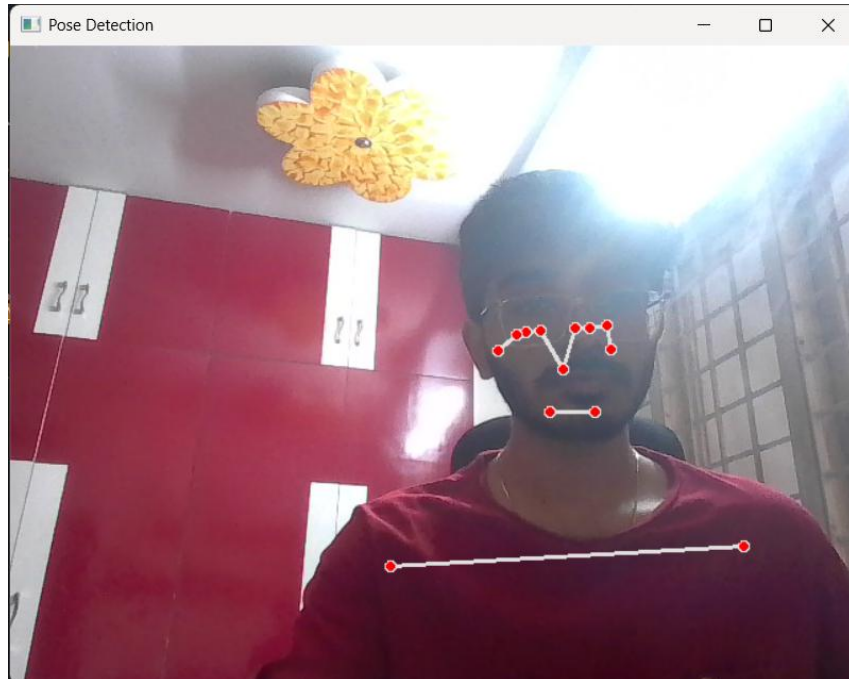
- Lighting
- Body pose
- Proper Camera Distance
- Visibility of the entire body

So, all this factors should be taken into account while trying to get the best results from a body detection system.

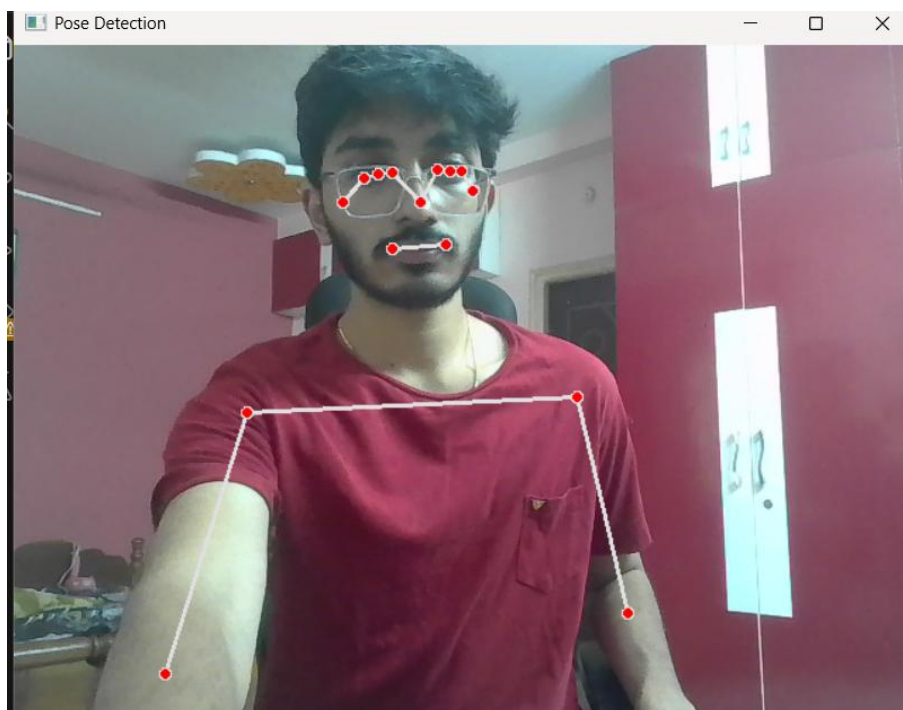
To test and improve its accuracy, it is important to train the model with different poses and lightings as well at different distances from camera to get the best detection.

So, lets check the body detection in different settings:

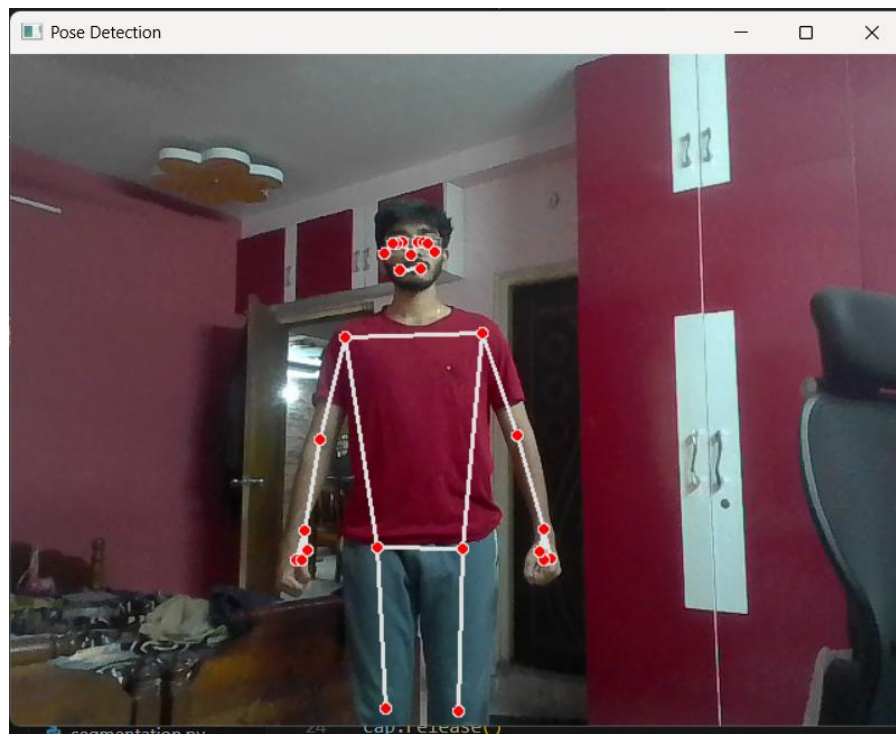
1) **BAD LIGHTING:**



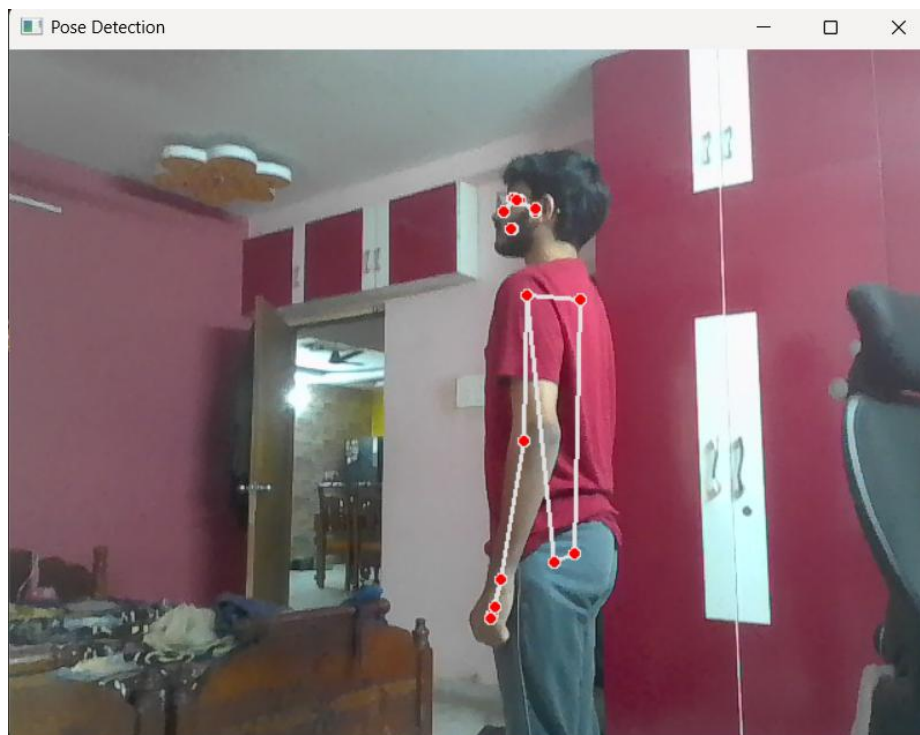
2) **GOOD LIGHTING**



3) STANDING STRAIGHT:



4) STANDING TO YOUR SIDE:



So in conclusion,

As we can see, The Body Detection system is most accurate when the person is standing straight and is in good lighting so this is the basic requirement from the customer to do this for the Raritone app to accurately detect their body entirely for Virtual Try-On and Best Size recommendation.

For the almost perfect body detection, you need other physical hardware like multiple cameras and deep camera that cannot be achieved by an mobile application that is helping with Virtual Try-On through your web or PC camera.

We have to understand that the application we are building won't provide the measurements right to its millimeters but it will be accurate enough to use for Virtual Try-On so the customer can experience how a clothing looks on them and which size is perfect for their body size.

MODULE : 2

BODY MEASUREMENTS USING POSE DETECTION

CODE USED FOR DEMONSTRATION

```

1 import cv2
2 import mediapipe as mp
3 import math
4 from tkinter import Tk
5 from tkinter.filedialog import askopenfilename
6
7 real_height_cm=float(input("Enter customer height in cm: "))
8
9 Tk().withdraw()
10
11 image_path=askopenfilename(
12     title="Select Customer Image",
13     filetypes=[("Image Files","*.jpg *.jpeg *.png")]
14 )
15
16 image=cv2.imread(image_path)
17
18 if image is None:
19     print("Image not loaded.")
20     exit()
21
22 image=cv2.resize(image,(900,700))
23
24 mp_pose=mp.solutions.pose
25 mp_draw=mp.solutions.drawing_utils
26
27 pose=mp_pose.Pose(
28     static_image_mode=True,
29     model_complexity=2,
30     min_detection_confidence=0.7
31 )
32
33 rgb=cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
34 results=pose.process(rgb)
35
36 def distance(p1,p2):
37     return math.sqrt((p2[0]-p1[0])**2+(p2[1]-p1[1])**2)
38
39 def point(lm,w,h):
40     return (lm.x*w,lm.y*h)
41
42 def draw_text(text,y,color):
43
44     cv2.putText(
45         image,
46         text,
47         (20,y),
48         cv2.FONT_HERSHEY_SIMPLEX,
49         0.7,
50         color,
51         2
52     )
53
54 if results.pose_landmarks:
55     landmarks=results.pose_landmarks.landmark
56     h,w=image.shape
57
58     mp_draw.draw_landmarks(
59         image,
60         results.pose_landmarks,
61         mp_pose.POSE_CONNECTIONS
62     )
63
64     for lm in landmarks:
65         x=int(lm.x*w)
66         y=int(lm.y*h)
67         cv2.circle(image,(x,y),4,(0,255,0),-1)
68
69     p={
70         "ls":landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER],
71         "rs":landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER],
72         "lh":landmarks[mp_pose.PoseLandmark.LEFT_HIP],
73         "rh":landmarks[mp_pose.PoseLandmark.RIGHT_HIP],
74         "le":landmarks[mp_pose.PoseLandmark.LEFT_ELBOW],
75         "lw":landmarks[mp_pose.PoseLandmark.LEFT_WRIST],
76         "lk":landmarks[mp_pose.PoseLandmark.LEFT_KNEE],
77         "la":landmarks[mp_pose.PoseLandmark.LEFT_ANKLE],
78         "ra":landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE],
79         "nose":landmarks[mp_pose.PoseLandmark.NOSE]
80     }
81
82     shoulder_pixels=distance(
83         point(p["ls"],w,h),
84         point(p["rs"],w,h)
85     )
86
87     waist_pixels=distance(
88         point(p["lh"],w,h),
89         point(p["rh"],w,h)
90     )
91
92     sleeve_pixels=distance(
93         point(p["ls"],w,h),
94         point(p["le"],w,h)
95     )+distance(
96         point(p["le"],w,h),
97         point(p["lw"],w,h)
98     )
99
100     torso_pixels=distance(
101         point(p["ls"],w,h),
102         point(p["lh"],w,h)
103     )
104
105     leg_pixels=distance(
106         point(p["lk"],w,h),
107         point(p["la"],w,h)
108     )+distance(
109         point(p["lk"],w,h),
110         point(p["la"],w,h)
111     )
112
113     avg_ankle_y=((p["la"].y+p["ra"].y)/2)*h
114
115     body_height_pixels=abs((p["nose"].y*h)-avg_ankle_y)
116
117     scale_factor=real_height_cm/body_height_pixels
118
119     measurements={
120         "Shoulder width":shoulder_pixels*scale_factor,
121         "Waist width":waist_pixels*scale_factor,
122         "Sleeve length":sleeve_pixels*scale_factor,
123         "Torso length":torso_pixels*scale_factor,
124         "Leg length":leg_pixels*scale_factor
125     }
126
127     lines=[
128         (p["ls"],p["rs"],(255,0,255)),
129         (p["lh"],p["rh"],(0,255,255)),
130         (p["ls"],p["le"],(0,0,255)),
131         (p["le"],p["lw"],(0,0,255)),
132         (p["ls"],p["lh"],(255,255,0)),
133         (p["lh"],p["lk"],(0,255,0)),
134         (p["lk"],p["la"],(0,255,0))
135     ]
136
137     for p1,p2,color in lines:
138         cv2.line(
139             image,
140             (int(p1.x*w),int(p1.y*h)),
141             (int(p2.x*w),int(p2.y*h)),
142             color,
143             3
144         )
145
146     colors=[
147         (255,0,255),
148         (0,255,255),
149         (0,0,255),
150         (255,255,0),
151         (0,255,0)
152     ]
153
154     for i,(name,value) in enumerate(measurements.items()):
155         draw_text(
156             f"{name}: {value:.1f} cm",
157             40+(i*40),
158             colors[i]
159         )
160
161     print("\nMeasurements:\n")
162
163     for name,value in measurements.items():
164         print(f"{name}: {value:.2f} cm")
165

```

```

162
163     for name,value in measurements.items():
164         print(f"{name}: {value:.2f} cm")
165
166     cv2.imshow("RARITONE Body Measurement System",image)
167
168     cv2.waitKey(0)
169     cv2.destroyAllWindows()
170
171 else:
172     print("Pose not detected properly.")

```

This code is used to measure different body lengths of a human body to get the perfect size for the customer.

THE OUTPUT FOR THIS CODE IS:



```

(venv) PS C:\Users\Akshith Kalvakota\Desktop\pose detection> python pd.py
Enter customer height in cm: 175
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.

Measurements:

Shoulder width : 52.24 cm
Waist width    : 28.20 cm
Sleeve Length : 62.60 cm
Torso Length  : 64.64 cm
Leg Length    : 89.84 cm
(venv) PS C:\Users\Akshith Kalvakota\Desktop\pose detection>

```

This code provides us the:

- Shoulder width
- Waist Width
- Sleeve Length
- Torso Length
- Leg Length

But first the user has to mention his/her height in cm so that conversion formulae can be used to convert the pixels length into actual human used measurements like centimeters.

LOGIC BEHIND IT:

1) The height of the user is taken.

2) Then the pose detection using MediaPipe detects the landmarks on the body and assigns a coordinate(x,y) to each landmark.

3) The distance between different landmarks is calculated to get the requirement measurements. For example, for shoulder width we take the distance between the left shoulder and the right shoulder.

The distance is taken using the Euclidean distance formula:

For (x,y) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

4) Now the distance taken is in pixels, so we use the real height of the user in cm to calculate the scale factor:

$$\text{Scale Factor} = \frac{\text{Real Height in cm}}{\text{Detected Height in Pixels}}$$

5) Now we convert each body measurement into cm using, for example:

$$\text{SHOULDER WIDTH (CM)} = \text{SHOULDER PIXELS} \times \text{SCALE FACTOR}$$

And that is how we get the final measurements in cm.